Project #3
Recursion & Binary Trees

Instructions

- This project will test your practical understanding of recursion as well as your ability to work with Shaffer's implementation of the **Binary Tree** and **Binary Search Tree** data structures
- The first sprint is due at **end of class Thursday 11/05/26**
- The complete project is due at **EOC Monday 11/16/26**
- DO NOT WAIT UNTIL THE LAST MINUTE!
- If you have any questions, ask me sooner rather than later
- If you run into algorithmic questions or syntactical problems, you should research - but all work that you submit must be your own
- In accordance with the HRS Generative AI Policy, this project may use "AI Planning"
    - YOU MUST SUBMIT THE CHAT LOG

Instructions: Create the Project

- Download the Project3 ZIP file from the class webpage and import it into NetBeans (File->Import Project)
- Rename the project to include your name (e.g."Project3-MBeck")
- The *BST* class includes a *toString()* method and associated methods that enables you to easily print out the contents of a BST using an in-order enumeration
- I've included a test harness that creates a BST of 30 random integers ranging from 0-300, then calls the first two methods you'll be implementing for this project
- Make sure the project compiles and runs
    - At this point you should see a printout of the BST, but nothing for the two method calls

Instructions: Task #1

- At the end of the ***BST* class**, you will find a method declaration for a function called *printHighToLow()*
- Implement this method as a recursive function that prints out the node **keys** of the BST in reverse sorted order from **highest** to **lowest**
- Add any additional helper methods that you may need in order to implement this functionality
- Do **not** alter any of the other existing BST methods!
- Once you have finished your implementation, run the project and confirm that your code does in fact do what you expect it to do

Instructions: Task #2

- At the end of the **test harness file**, you will find a method declaration for a function called *printRange()*
  - We are defining this function in the test harness because at this stage, to simplify our code, we are going to assume that our BST consists of nodes with *Integer* keys
- Implement this method as a recursive function that, given a pointer to the root node of a BST, a low key integer value, and a high key integer value, prints in sorted order all records whose **key values** fall between the two given keys, **inclusive**
- The function should visit as few nodes in the BST as possible
- Once you have finished your implementation, run the project and confirm that your code does in fact do what you expect it to do

Instructions: Task #3

- In the **test harness file**, create a method declaration for a function called *sumNodes()*
  - Similar to the above, we are defining this function in the test harness because we are going to create the BST to consist of nodes with *Integer* elements
- Implement this method as a recursive function that, given a pointer to the root node of a BST storing integer elements in its nodes, returns the sum of the **elements** of all nodes in the tree
- Update the test harness *main()* method to test and print the results of your function
  - Note that your method needs a pointer to the root node of the BST: if needed, refer to how *printRange()* is called

Instructions: Task #4

- In the **test harness file**, create a method declaration for a function called *smallCount()*
  - Similar to the above, we are defining this function in the test harness because we are going to create the BST to consist of nodes with *Integer* keys
- Implement this method as a recursive function that, given a pointer to the root node of a BST storing integer values in its nodes and a key value K, returns the number of nodes having **key values** less than or equal to K
- The function should visit as few nodes in the BST as possible
- Update the test harness *main()* method to test and print the results of your function

Hints

- Start with your stopping condition. Make sure you understand when the recursive traversal should end!
- Handle the empty tree case. Your stopping condition likely should be when (*rt* == null). Don't dereference a null pointer!
- Understand what your operations are (e.g. sum, count, print?)
- Understand possible states for every call/node. Do you do different operations when e.g. the node key value < *K*?
- Call your recursive function as if it works, and only deal with a node at a time: for instance, if you're summing nodes, sum the node you're on, the left subtree, and the right subtree
- Make sure you're returning a value for every call, and that you handle every value that's returned

END OF SPRINT #1
STOP HERE

Instructions: Task #5

- In the ***BST* class**, create a method declaration for a function called *countNodes()*
- Implement this method as a recursive function that returns the count of all the nodes in the tree
- Update the test harness *main()* method to test and print the results of your function

Instructions: Task #6

- In the **test harness file**, create a method declaration for a function called *evenNodes()*
  - Similar to the above, we are defining this function in the test harness because we are going to create the BST to consist of nodes with *Integer* elements
- Implement this method as a recursive function that, given a pointer to the root node of a BST storing integer elements in its nodes, returns the count of all the nodes with even **elements** in the tree
- Update the test harness *main()* method to test and print the results of your function

Instructions: Task #7

- In the ***BST* class**, create a method declaration for a function called *countLeaves()*
- Implement this method as a recursive function that returns the count of all the leaves (nodes with only null children) in the tree
- Update the test harness *main()* method to test and print the results of your function

Instructions: Task #8

- In the ***BST* class**, create a method declaration for a function called *countParents()*
- Implement this method as a recursive function that returns the count of all the parents (nodes with at least one non-null child) in the tree
- Update the test harness *main()* method to test and print the results of your function

Instructions: Extra Credit #1

- In the ***BST* class**, create a method declaration for a function called *printInLevel()*
- Implement this method as a function that prints the node **elements** of the tree in in-level order (print the root node, then all the nodes in level 1 left-to-right, then all the nodes in level 2 left-to-right, and so on)
- Update the test harness *main()* method to test and print the results of your function
- HINT: you may want to use a FIFO data structure to track which elements need to be printed. Try to understand the operation using pennies laid out on a table, navigating as if you are a computer!

Instructions: Extra Credit #2

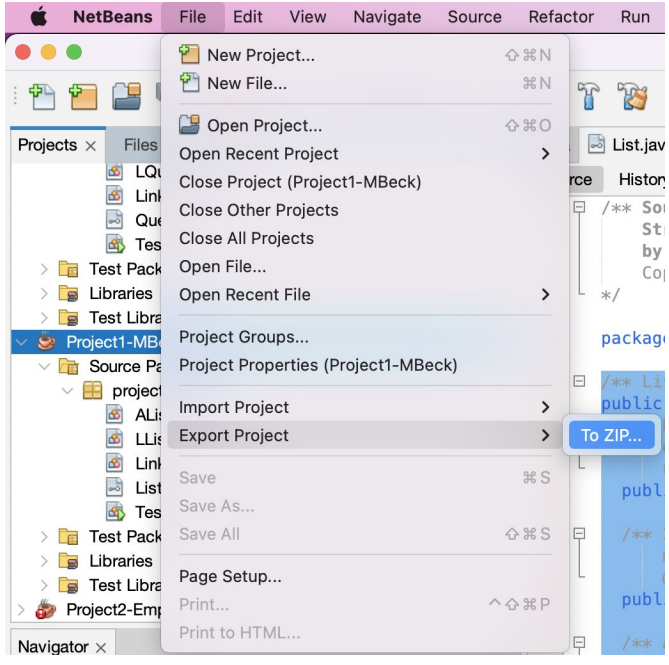
- In your **test harness**, create a BST that stores *Character* keys and elements, instead of integers
- Figure out how to generate random characters, and store them in your new BST, using the same char for both key and element
- Run *printHighToLow()* on your new character BST
- Consider that the *Character* class implements the *Comparable* interface, so it supports calling the *compareTo()* method
- Move the *printRange()* method into your *BST* class, and re-implement it using the *compareTo()* method, so it's no longer dependent on integer keys
- Call the method in your test harness and verify it prints a subset of your character BST, and still works for your integer BST

Grading

- Full credit will be given if all of the following criteria are met:
  - Your project compiles
  - Your code is clean and is documented correctly
  - You create the test harness and test your code across all edge cases
  - Your new code executes correctly when dropped into my test harness
- Partial credit will be given if only some of the criteria are met
- Test your code before submitting!

Submission

- To submit your code on HRSConnect, you must **export** it
- Highlight your project in the Projects browser on the left pane, then click the "File" menu -> "Export Project" -> "To Zip..."



Submission

- Click on the "Browse" button next to the "Build ZIP:" field and select a convenient location for your export file (e.g. Desktop)
- Hit the "Export" button and upload the resulting file to HRSConnect

